Real-time Water Meter Reading Based on TinyML

Van-Khanh Nguyen
Faculty of Automation Technology
Can Tho University
Can Tho, Vietnam
vankhanh@ctu.edu.vn

Vy-Khang Tran
Faculty of Automation Technology
Can Tho University
Can Tho, Vietnam
khangm3523001@gstudent.ctu.edu.vn

Bao-Toan Thai
Faculty of Automation Technology
Can Tho University
Can Tho, Vietnam
toanb1912990@student.ctu.edu.vn

Hai Pham
Faculty of Automation Technology
Can Tho University
Can Tho, Vietnam
ptlhai@ctu.edu.vn

Muot Nguyen
Faculty of Automation Technology
Can Tho University
Can Tho, Vietnam
nvmuot@ctu.edu.vn

Truong Quoc Bao
Faculty of Automation Technology
Can Tho University
Can Tho, Vietnam
tqbao@ctu.edu.vn

Chi-Ngon Nguyen
Faculty of Automation Technology
Can Tho University
Can Tho, Vietnam
ncngon@ctu.edu.vn

***Abstract*— This paper proposes a system that automatically reads the value of the water meter using a tiny machine learning model (tinyML) running directly on the ESP32-CAM kit. A convolutional neural network (CNN)-based tinyML model is recommended to recognize meter digits before estimating a value. The digit recognition machine learning (ML) models were trained using three datasets generated from this study, including a grayscale image set, a contrast-enhanced image set using the HE algorithm, and a binary image set using the threshold determined by the adaptive threshold (AT) algorithm to find the matching set of images. The experimental results show that the proposed classification model with the input gray image and contrast-enhanced image gives the best accuracy of 98.3%, and the estimated speed is approximately 3 times per second. This accuracy is approximate compared to the previous study; however, the image data processing solution in this study provides roughly 10 times faster estimation time. Furthermore, the study shows that gray images should be used directly for the digit classification problem instead of being contrast-enhanced or converted to binary images.**

Keywords— Meter reading; Tiny machine learning; ESP32 CAM; Grayscale image.

I. INTRODUCTION

Currently, water meters are being widely used in many countries around the world. Especially in developing countries like Vietnam still using a lot of mechanical water meters, switching to digital meters is expensive, so most households use mechanical water meters. This results in high costs for the monthly metering team. Therefore, researchers are aiming for automation, creating an automatic system to read water meter readings, which is very useful in motels, manufacturing companies, etc. The system helps households to easily control household water control, saving costs incurred from manual water readings and water leak detection. This is very necessary when the world is facing climate change, sea level rise makes clean water sources increasingly narrow.

Some previous methods applied computer vision (CV) combined with artificial intelligence to recognize indicators of water meters, electricity meters, etc. Naim, et al. [1] developed an automatic water meter reading application, a convolutional neural networks (CNN) model trained using the Moroccan Automatic Meter Reading (MR-AMR) dataset to recognize digits to calculate values water index, and a web-based service also developed to be made available to users [1]. Several deep learning models that are Faster R-CNN, SSD, YOLOv3, and YOLOv5, recurrent neural network (RNN) have also been applied to automatic reading of water meter readings [2-4]. In addition to these models, YOLOv4 has also been applied to read the value of electric meters [5, 6], and pointer meters [7]. Currently, there are many water meters located in different environments, which makes it difficult to read the value automatically. Hong, et al. [8] proposed an automatic water reading system fully CNN-based including the following steps, water meter detection and rotation-corrected component localization, regression-based digit reading with spatial layout guidance, and pointer reading. Most of the proposed models are complicated with many parameters, suitable for training and execution on personal computers, and difficult to implement in practice. Therefore, finding a functionally equivalent machine learning model that can be deployed on microcontrollers with limited memory and computational resources will be essential for integration into real-world devices.

Based on these methods, Optical character recognition (OCR) technique is also applied to get each reading of a water meter by identifying a series of numbers on the front of the water meter. [9, 10] used the OCR method to read water index automatically. However, Jawas [9] performed processing operations on a computer. By contrast, Duc, et al. [10] applied this method on the ESP32-CAM circuit. A convolutional neural network (H-CNN) model with 40 output units was proposed to read the digits 0 to 9 on each digit of the water meter. This model was trained using a binary image dataset collected by the built-in camera. Although the index reading result is more than 98 %, this study used complicated image preprocessing steps. The author had to take an RGB image and then convert it to gray before applying image processing algorithms such as blur, histogram equalization, binarize, erode, and crop (auto) to create binary images for the digits to

979-8-3503-5531-4/24/$31.00 ©2024 IEEE

be collected. Therefore, it takes a long time to complete a read operation, leading to an increase in system power consumption. However, for the CNN network, it integrates layers that specialize in extracting features of groups of pixels on images, so converting to binary images for training and recognition may not be necessary. Furthermore, for data of numbers from 0 to 9, using 40 output units is not optimal, it will increase the complexity of the network model.

TinyML enables energy-efficient, low-cost, and reliable machine learning on microcontroller units (MCUs), allowing smart devices to perform real-time data processing directly on the device. This minimizes power consumption, reduces latency, and enhances data security, making it ideal for IoT applications across various sectors [11]. This paper aims to propose a tinyML machine learning model based on a CNN network with 10 output units (i.e., recognizing digits from 0 to 9) to support automatic reading of water meter values. The proposed tinyML model is trained using three sets of image data collected by the integrated camera, including a set of live grayscale images; a set of images after contrast enhancement by the histogram equalization (HE) algorithm; and a set of binary images whose threshold is determined by the adaptive threshold (AT) algorithm. The post-trained tinyML models were executed on the ESP32-CAM circuit to read the water meter value with real-time snapshots from the built-in camera. Experimental results also aim to determine the most optimal image type for the system.

II. METHODOLOGY

The system is designed through two main stages as illustrated in Fig. 1 including training of the numerical recognition model based on tinyML and deploying trained models to estimate the meter's value. Phase 1 includes collecting image data sets using the built-in camera on the ESP32-CAM kit and training digital recognition models on Google Colab. The gray image dataset of digits 0 to 9 was collected directly on the microcontroller, sent to the computer for storage and enhanced to ensure the completeness of the data. The grayscale image dataset is then contrast-enhanced and binary to generate two more datasets for training the recognition models. In the second phase, post-training models were applied to recognize the digits on the meter through the camera's direct image to estimate the meter's value.

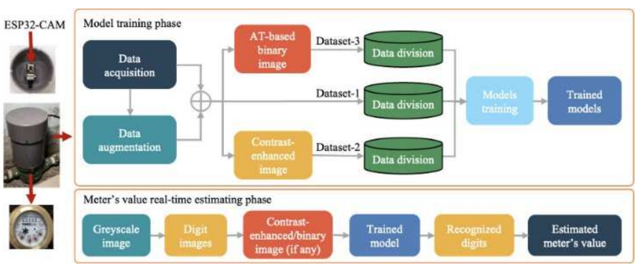


Fig. 1 - Overview of implementation methods.

The estimation accuracy was calculated to determine the most appropriate model and image type. The HE image and binary image are correspondingly implemented on the computer and the MCU that should yield similar results.

A. Algorithm to crop with numbers on the image

The image data of the numbers were collected directly using the ESP32-CAM kit's built-in imaging computer. The camera was fixed on the lid of the device at 12 cm from the meter. The camera's focus is adjusted manually for the clearest picture. The control program takes a gray image of size 640×480 directly instead of taking a color image and then converting it to a gray image as in [10]. This greatly reduces the processing time. Next, the image of the digits on the water meter value was cropped based on the coordinates of the origin points *AOi* of each image as shown in Fig. 2. From these coordinates, the four origin coordinates of the images were determined by the following formula:

$$AOi = \begin{cases} \left(x,\ x,\ x + \Delta x,\ x + \Delta x\right) \\ \left(y_i,\ y_i + \Delta y,\ y_i,\ y_i + \Delta y\right) \end{cases} \quad (1)$$

Where: Δx=32 and Δy=18; i is from 1 to 4, corresponds to images from thousands to units. Once these coordinates are known, the images can be easily cropped using an algorithm as illustrated in Algorithm 1. The image size is 32×18 contains the exact numbers on the meter. Since the snapshot is stored on a 1-dimensional array of size 1 × (480×640), the algorithm uses four iterations, the k loop determines the order of images to be taken, the i loop determines the i^{th} row of k images, and the j loop defines column j of image k. The resulting images are stored in a two-dimensional array *Img* for data collection or meter value estimation.

Algorithm 1. Crop image

Input: a single image with *fb* ← esp_camera_fb_get();
Output: An array *Img* [4][576]
x ← coordinates defined
Δx = 32 and Δy = 18
L [4] ← 4 specified coordinate points
for *k* ← 0 to 4 do:
    pixel ← 0
    for *i* ← *x* to *x* + Δx do:
        for *j* ← *L*[*k*] to *L*[*k*] + Δy do:
            // accessing the image data buffer
            Img[*k*][*pixel*]←(*fb*->buf[(*i* - 1)*640 + *j*]-128)
            pixel ++   // run from 0 to 576
        end for
    end for
end for

B. Data Augmentation and Preprocess

1) Data Augmentation: Initial tests show that only training the recognition model using grayscale images results in quite large overfitting outputs. This can be caused by reasons such as splitting random datasets that cause imbalances, out-of-angle or pixelated photography angles. Data enhancement methods including random rotations [12], and random shifts [13] are applied to generate more data from the original grayscale image set, and the dataset is more general. The rotation angle of the random rotation algorithm is limited to 15 degrees, random shift only applies random up/down shifts. Data augmentation not only helps to reduce overfitting during training, but it also helps to limit the effect of a small offset of the camera housing cover.

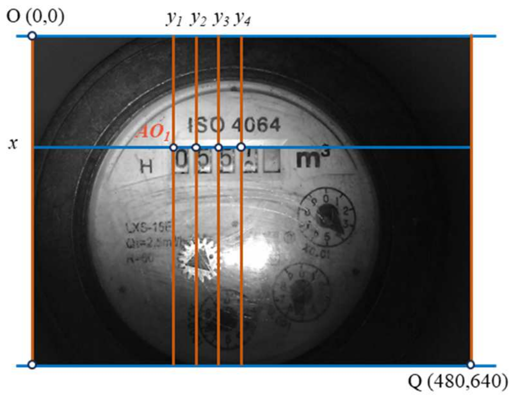


Fig. 2. The coordinates of the image origin of the digits on the meter.

2) Preprocessing data: Two algorithms HE [14] and AT [15] were applied to generate two additional datasets from the collected and enhanced grayscale images. These datasets are applied to train and deploy the model to find the right image type to help the system have high numerical estimation accuracy and low total processing time.

a) Histogram equalization: The HE algorithm (HE) is used to enhance the contrast of the original grayscale image. This algorithm is usually applied before converting an image to a binary image. Let I be the image presented in the form of a matrix of size $mr \times mc$. Let p denote the normalized histogram of I with one bin for each intensity. The values of p will be calculated according to the following formula:

$$p_n = \frac{number\ of\ \ pixels\ with\ intensity\ n}{total\ number\ of\ \ pixels} \tag{2}$$

Where n = 0, 1, …, L–1, L is the total number of gray levels in the image (a total of 256 levels from 0 to 255). The histogram equalized image g is calculated by the following formula:

$$g(i,\ j)\ = floor\left((l-1)\sum\nolimits_{n=0}^{f_{i,j}} p_n\right) \tag{3}$$

Where *floor()* is the rounding to the nearest integer. This algorithm will also be implemented on the MCU as shown in Algorithm 2 for image processing of digits when estimating real-time values. HE image was built on the MCU in C language.

b) Adaptive Threshold Algorithm: AT algorithm is the method used to determine the threshold for converting a grayscale image to a binary image. Otsu proposed a solution that automatically determines the threshold when converting a gray image to binary [15]. Accordingly, a gray image is represented by L gray levels [1, 2, …, L], between-class variance between two classes (background and objects, or vice versa) C_0 includes pixels with threshold [1, ..., k] and C_1 consist of pixels with the threshold [k+1, …, L] determined by the following formula:

$$\sigma_B^2(k) = \frac{[\mu_T\omega(k) - \mu(k)]^2}{(\omega(k)[1-\omega(k)])} \tag{4}$$

Where:

$$\mu_T = \sum_{i=1}^{L} ip_i \tag{5}$$

$$\mu(k) = \sum_{i=1}^{k} ip_i \tag{6}$$

$$\omega(k) = \sum_{i=1}^{k} p_i \tag{7}$$

are the total mean level of the original picture; zeroth- and the first-order cumulative moments of the histogram up to the k^{th} level, respectively. The optimal threshold k^* is determined by the following equation (8) as shown below. AT is also established on the MCU using the C language.

$$\sigma_B^2(k^*)\ =\ \max_{1\leq\ k<L}\ \sigma_B^2(k) \tag{8}$$

C. TinyML model design

The system was developed to simulate human reading of numbers. A machine learning model will be applied to read the numbers in the units, tens, hundreds, and thousand rows on the water meter, respectively, before recombining them into a readable value. Fig. 3 shows tinyML models based on the CNN network proposed to train the number recognition on the water meter. The model input is a 32×18 image containing numbers, and the output consists of 10 neurons containing the identification probabilities of numbers from 0 to 9. The proposed network consists of two main blocks including feature extraction and recognition. Feature extraction block includes 3 convolution layers and 3 max pooling; The classification block consists of 2 dropout layers, 1 flatten, and 3 dense layers with the last Dense layer using softmax activation function. Two dropout layers were used instead of one to reduce overfitting more effectively. The proposed model has a total of 81 466 training parameters, suitable for implementation in the MCU as stated by Nguyen, et al. [16].

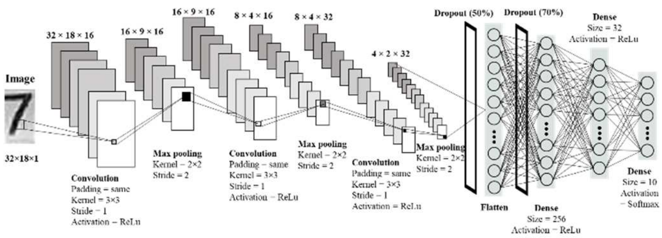


Fig. 3. The proposed digits classification model.

D. Real-time assessment method

Post-training classification models will be performed on the ESP32 to read the value on the water meter. The accuracy of the readings will be the basis for evaluating the models rather than relying on the classification of individual digits. Fig. 4 shows the program algorithm flowchart on ESP32. The program is an endless loop that reads the water index value. A loop will perform three main steps including taking a grayscale image of the meter to image the digits and applying HE image or binary image (if necessary); reclassifying the digits using trained models; aggregate the identifiers into a readable value. Assuming the units, tens, hundred, and thousand numbers on the meter are u, d, h, and t respectively; the meter's value, *Est*, will be calculated according to the following formula:

$$Est = 1000t + 100h + 10d + u \qquad (10)$$

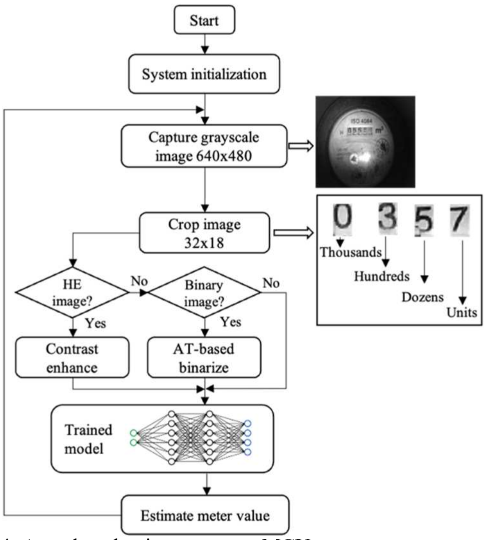


Fig. 4. Actual evaluation stages on MCU.

The estimation accuracy of meter values is evaluated after 1800 cases of estimating numbers from 0 to 299. Each model was applied to read each value in three main cases including two images of visual units with clear numbers, two images intersect numbers in the first stage and 2 images intersect numbers at the last stage or prepare for new numbers. Crossovers are cases where one number on the clock moves to the next. Fig. 5 illustrates three cases of reading zero on the meter. In the process of reading tens and hundred numbers can also be in the number intersection state.

a) Case 1 b) Case 2 c) Case 3

Fig. 5. Cases for estimating a value on a meter.

III. Result

A. Acquired Datasets

Three datasets were created to train the digit classification models including grayscale images collected from ESP32-CAM, and two datasets transformed by HE image and binary image. The HE image and binary image also need to be installed on the MCU for real-time classification of numbers, so the correct implementation of these algorithms is very important. Correlation coefficient R calculated by the formula (9) [17] is used to image the results on MCU and computer. Fig. 6 shows the histogram and cumulative histogram results of the algorithm on the MCU and PC. These results show that the MCU algorithm has equivalent results to PC; the R value of histogram equalization and cumulative histogram graph between MCU and PC reached 99.9%. The binary image converted by AT algorithm also achieved a very good correlation between MCU and PC as shown in Fig. 7, R also reached 95.5%. Thus, the HE image and binary image that have been successfully installed on the MCU can be applied to process input images for digital classification networks.

$$R = \frac{\sum \left(x_i - \bar{x}\right)\left(y_i - \bar{y}\right)}{\sum \left(x_i - \bar{x}\right)^2\left(y_i - \bar{y}\right)^2} \cdot 100 \qquad (11)$$

Where:

R = correlation coefficient [%],
x_i = values of the x variable in sample,
$\bar{x}$ = mean of the x variable,
y_i = values of the y variable in sample,
$\bar{y}$ = mean of the y variable.

The collected data sets contain 4800 samples including camera acquired and augmented images. In each data set, the images are stored in folders labeled with numbers from 0 to 9. Before use for training, the data sets will be randomly divided into two small sets including training and practice validation with a ratio of 80% and 20%, respectively. In this study, the post-training model will be tested directly through its application to the water meter index estimation. Fig. 8 shows some samples in the data sets.

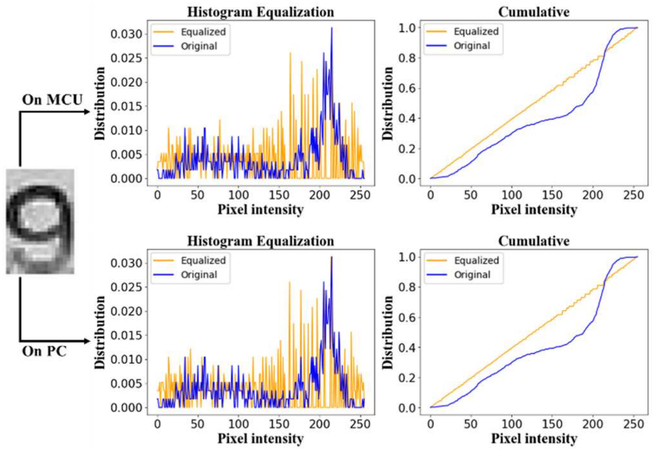
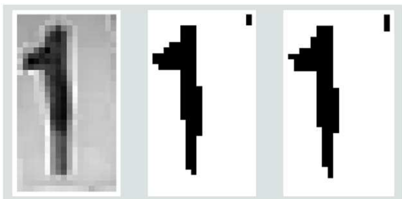


Fig. 6. Transformation results of histogram equalization algorithm on MCU and PC.



a) b) c)

Fig. 7. Binary image conversion result applying AT: a) grayscale image, a) executing on MCU, and c) executing on PC.

a) b) c)

Fig. 8. Data sets are included in the model for training.

B. Training results on computer model

Fig. 9 shows the loss and accuracy of the proposed model when trained on a computer using the three acquired datasets. For the proposed model, the overfitting was minimized to an acceptable level. The gray image dataset gives training results with the least overfitting. Table 1 presents the training accuracy of the computer models.

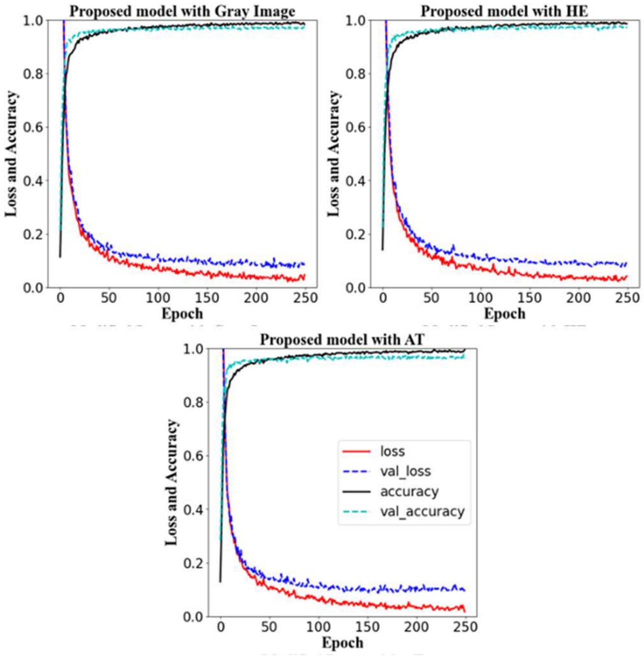


Fig. 9. Accuracy and loss of three datasets trained on three models.

TABLE 1. RESULT TRAINING ON COMPUTER

Dataset	Training accuracy (%)	Validation accuracy (%)
Gray Image	98.3	97.5
Histogram equalization	98.6	96.9
Adaptive threshold	99.4	96.3

C. Actual results on MCU

The real-time reading accuracy of values from 0 to 299 from the meter at 1800 is shown in Table 2. The results demonstrate that the readings are most accurate 98.3% when the classification model is applied. The proposed number was trained by a gray image dataset and an enhance-contrast image dataset by a histogram equalization algorithm. However, more than 300 ms is still highly efficient for an automatic digit reading system. Moreover, Table 2 also shows that using the set can directly use the gray image set to train the digit classification model without necessarily further processing as in [10]. This helps to speed up reading the value by not having to apply additional image preprocessing algorithms.

TABLE 2. EXPERIMENTAL RESULTS

Dataset	Accuracy (%)	Processing time (ms)
Gray Image	98.3	311.6
Histogram equalization	98.3	313.3
Adaptive threshold	96.6	321.5

IV. DISCUSSIONS

Thus, the automatic reading system of the water meter based on tinyML has been successfully implemented with an accuracy of more than 98% equivalent to the results of the previous study by Duc, et al. [10]. The model was trained by the dataset collected right on the system, so it was active in the design process. Besides, the paper also proved that the gray image dataset is perfectly suitable to train the tinyML model for the digit classification problem without having to go through the quality improvement algorithms or convert it to a binary image. In addition, the data collector has successfully trained metered number recognition CNNs with only 10 or more output neurons to deal with number crossings.

The estimated time from taking an image to printing the result is very short, only about 312 ms. This speed is achieved because we capture grayscale images directly and bypass some image preprocessing algorithms. Indeed, in the same study, Duc, et al. [10] took a JPEG image and converted it to a grayscale image before applying a variety of preprocessing operations to form the binary image. The separate statistics of the time taken to capture and process images taken by Duc et al took 3.13 s compared to 10.9 ms in this study. The single digit recognition time of the proposed model in this study is approximately twice as long as that of Duc, et al. [10] due to its much larger deep levels. However, the total time from imaging to estimating the value of this research meter took only 321.8 ms, 10.7 times faster than the system of Duc et al.

The proposed model estimates an incorrect value of 1.7%, but the main reason comes from the mechanical structure of the meter. Some cases of erroneous system estimations are illustrated in Fig. 10. Mistakes occur when the model does not correctly classify the numbers in the case of intersections, leaving the next number to appear higher such as collected data, it is lower due to mechanical error. This can be overcome by using some water meters with mechanical accuracy, or by adding an error correction algorithm to the rate of change of the value on the meter.

TABLE 3. COMPARES THE PROCESSING TIME OF THE PROPOSED MODEL WITH THE RESEARCH MODEL OF DUC ET AL.

Task	Duc et al. (ms)	This paper (ms)
Capture image	140	0.02
Convert image frame to RGB888	1670	-
Convert image to grayscale	390	-
Crop out zone around digit area	110	-
Rotate image	360	-
Crop out image of 4 digits; equalize histogram, binarize, erode 4 digits	600	10.90
Make inference for 4 digits	160	310.88
Total processing time	3430	321

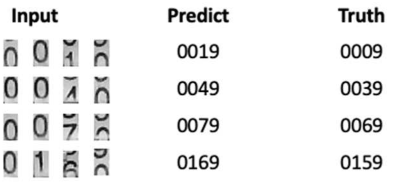


Fig. 10. Model number of falsely identified samples on the proposed model.

V. CONCLUSIONS

This study presented a full solution to automatically read the value of water meter based on tinyML network. An operational model based on ESP32-CAM kit with built-in camera mounted directly on water meter. The CNN-based tinyML model was trained using three datasets collected by this study. The model was used for real-time estimation of the numbers from 0 to 299 on the meter in 1800 in both clear-sighted and cross-numbered cases. The results confirm the accuracy of the proposed model for meter water value estimation up to 98.3% when input is the grayscale and contrast-enhanced image, which takes only approximately 300 ms. Besides, the tinyML network suggests using fewer output neurons than the prior research model, so its depth can be adjusted to improve efficiency. Moreover, the study also contributes to proving that the gray image dataset can be used directly to train the CNN network for the classification problem instead of having to be binary, as in previous studies.

REFERENCES

[1] A. Naim, A. Aaroud, K. Akodadi, and C. El Hachimi, "A fully AI-based system to automate water meter data collection in Morocco country," Array, vol. 10, p. 100056, 2021.

[2] Y. Liang, Y. Liao, S. Li, W. Wu, T. Qiu, and W. Zhang, "Research on water meter reading recognition based on deep learning," Scientific Reports, vol. 12, no. 1, p. 12861, 2022.

[3] F. Martinelli, F. Mercaldo, and A. Santone, "Water meter reading for smart grid monitoring," Sensors, vol. 23, no. 1, p. 75, 2022.

[4] J. Zhang, W. Liu, S. Xu, and X. Zhang, "Key point localization and recurrent neural network based water meter reading recognition," Displays, vol. 74, p. 102222, 2022.

[5] G. Salomon, R. Laroca, and D. Menotti, "Deep learning for image-based automatic dial meter reading: Dataset and baselines," in 2020 International Joint Conference on Neural Networks (IJCNN), 2020: IEEE, pp. 1-8.

[6] G. Salomon, R. Laroca, and D. Menotti, "Image-based Automatic Dial Meter Reading in Unconstrained Scenarios," Measurement, vol. 204, p. 112025, 2022.

[7] X. Wu, X. Shi, Y. Jiang, and J. Gong, "A high-precision automatic pointer meter reading system in low-light environment," Sensors, vol. 21, no. 14, p. 4891, 2021.

[8] Q. Hong et al., "Image-based automatic watermeter reading under challenging environments," Sensors, vol. 21, no. 2, p. 434, 2021.

[9] N. Jawas, "Image based automatic water meter reader," in Journal of Physics: Conference Series, 2018, vol. 953, no. 1: IOP Publishing, p. 012027.

[10] H. N. Duc, T. N. Manh, H. T. Le, and F. Ferrero, "Research and Implement Embedded Artificial Intelligence in Low-Power Water Meter Reading Device," in 2021 International Conference on Advanced Technologies for Communications (ATC), 2021: IEEE, pp. 119-124.

[11] R. Sanchez-Iborra and A. F. Skarmeta, "Tinyml-enabled frugal smart objects: Challenges and opportunities," IEEE Circuits and Systems Magazine, vol. 20, no. 3, pp. 4-18, 2020.

[12] R. Blaser and P. Fryzlewicz, "Random rotation ensembles," The Journal of Machine Learning Research, vol. 17, no. 1, pp. 126-151, 2016.

[13] C. Shorten and T. M. Khoshgoftaar, "A survey on image data augmentation for deep learning," Journal of big data, vol. 6, no. 1, pp. 1-48, 2019.

[14] Y. Wang, Q. Chen, and B. Zhang, "Image enhancement based on equal area dualistic sub-image histogram equalization method," IEEE transactions on Consumer Electronics, vol. 45, no. 1, pp. 68-75, 1999.

[15] N. Otsu, "A threshold selection method from gray-level histograms," IEEE transactions on systems, man, and cybernetics, vol. 9, no. 1, pp. 62-66, 1979.

[16] V. K. Nguyen, V. K. Tran, M. K. Nguyen, T. L. H. Pham, and C. N. Nguyen, "Realtime Non-invasive Fault Diagnosis of Three-phase Induction Motor," Journal of Technical Education Science, no. 72B, pp. 1-11, 2022.

[17] C. Huang et al., "An Intelligent Rice Yield Trait Evaluation System Based on Threshed Panicle Compensation," Frontiers in Plant Science, vol. 13, p. 900408, 2022.